

Equations (11.1.4) – (11.1.7) imply that

$$S' = S - 2|a_{pq}|^2 \quad (11.1.20)$$

(Since the transformation is orthogonal, the sum of the squares of the diagonal elements increases correspondingly by $2|a_{pq}|^2$.) The sequence of S 's thus decreases monotonically. Since the sequence is bounded below by zero, and since we can choose a_{pq} to be whatever element we want, the sequence can be made to converge to zero.

Eventually one obtains a matrix $\mathbf{D}$ that is diagonal to machine precision. The diagonal elements give the eigenvalues of the original matrix $\mathbf{A}$, since

$$\mathbf{D} = \mathbf{V}^T \cdot \mathbf{A} \cdot \mathbf{V} \quad (11.1.21)$$

where

$$\mathbf{V} = \mathbf{P}_1 \cdot \mathbf{P}_2 \cdot \mathbf{P}_3 \cdots \quad (11.1.22)$$

the $\mathbf{P}_i$'s being the successive Jacobi rotation matrices. The columns of $\mathbf{V}$ are the eigenvectors (since $\mathbf{A} \cdot \mathbf{V} = \mathbf{V} \cdot \mathbf{D}$). They can be computed by applying

$$\mathbf{V}' = \mathbf{V} \cdot \mathbf{P}_i \quad (11.1.23)$$

at each stage of calculation, where initially $\mathbf{V}$ is the identity matrix. In detail, equation (11.1.23) is

$$\begin{aligned} v'_{rs} &= v_{rs} & (s \neq p, s \neq q) \\ v'_{rp} &= cv_{rp} - sv_{rq} \\ v'_{rq} &= sv_{rp} + cv_{rq} \end{aligned} \quad (11.1.24)$$

We rewrite these equations in terms of τ as in equations (11.1.16) and (11.1.17) to minimize roundoff.

The only remaining question is the strategy one should adopt for the order in which the elements are to be annihilated. Jacobi's original algorithm of 1846 searched the whole upper triangle at each stage and set the largest off-diagonal element to zero. This is a reasonable strategy for hand calculation, but it is prohibitive on a computer since the search alone makes each Jacobi rotation a process of order N^2 instead of N .

A better strategy for our purposes is the *cyclic Jacobi method*, where one annihilates elements in strict order. For example, one can simply proceed down the rows: $\mathbf{P}_{01}, \mathbf{P}_{02}, \dots, \mathbf{P}_{0,n-1}$; then $\mathbf{P}_{12}, \mathbf{P}_{13}$, etc. One can show that convergence is generally quadratic for either the original or the cyclic Jacobi method, for nondegenerate eigenvalues. One such set of $n(n-1)/2$ Jacobi rotations is called a *sweep*.

The program below, based on the implementations in [1,2], uses two further refinements:

- In the first three sweeps, we carry out the pq rotation only if $|a_{pq}| > \epsilon$ for some threshold value

$$\epsilon = \frac{1}{5} \frac{S_0}{n^2} \quad (11.1.25)$$

where S_0 is the sum of the off-diagonal moduli,

$$S_0 = \sum_{r < s} |a_{rs}| \quad (11.1.26)$$

- After four sweeps, if $|a_{pq}| \ll |a_{pp}|$ and $|a_{pq}| \ll |a_{qq}|$, we set $|a_{pq}| = 0$ and skip the rotation. The criterion used in the comparison is $|a_{pq}| < 10^{-(D+2)}|a_{pp}|$, where D is the number of significant decimal digits on the machine, and similarly for $|a_{qq}|$.

Typical matrices require six to ten sweeps to achieve convergence, or $3n^2$ to $5n^2$ Jacobi rotations. Each rotation requires of order $8n$ floating-point operations, so the total labor is of order $24n^3$ to $40n^3$ operations. Calculation of the eigenvectors as well as the eigenvalues changes the operation count from $8n$ to $12n$ per rotation, which is only a 50% overhead.

The following routine implements the Jacobi method. Simply create a Jacobi object using your symmetric matrix `a[0..n-1][0..n-1]`:

```
Jacobi jac(a);
```

The vector `d[0..n-1]` then contains the eigenvalues of `a`. During the computation, it contains the current diagonal of `a`. The matrix `v[0..n-1][0..n-1]` outputs the normalized eigenvector belonging to `d[k]` in column `k`. The parameter `nrot` is the number of Jacobi rotations that were needed to achieve convergence.

eigen_sym.h

```
struct Jacobi {
    Computes all eigenvalues and eigenvectors of a real symmetric matrix by Jacobi's method.
    const Int n;
    MatDoub a,v;
    VecDoub d;
    Int nrot;
    const Doub EPS;

    Jacobi(MatDoub_I &aa) : n(aa.nrows()), a(aa), v(n,n), d(n), nrot(0),
        EPS(numeric_limits<Doub>::epsilon())
    Computes all eigenvalues and eigenvectors of a real symmetric matrix a[0..n-1][0..n-1].
    On output, d[0..n-1] contains the eigenvalues of a sorted into descending order, while
    v[0..n-1][0..n-1] is a matrix whose columns contain the corresponding normalized eigen-
    vectors. nrot contains the number of Jacobi rotations that were required. Only the upper
    triangle of a is accessed.
    {
        Int i,j,ip,iq;
        Doub tresh,theta,tau,t,sm,s,h,g,c;
        VecDoub b(n),z(n);
        for (ip=0;ip<n;ip++) {
            for (iq=0;iq<n;iq++) v[ip][iq]=0.0;
            v[ip][ip]=1.0;
        }
        for (ip=0;ip<n;ip++) {
            b[ip]=d[ip]=a[ip][ip];
            z[ip]=0.0;
        }
        for (i=1;i<=50;i++) {
            sm=0.0;
            for (ip=0;ip<n-1;ip++) {
                for (iq=ip+1;iq<n;iq++)
                    sm += abs(a[ip][iq]);
            }
            if (sm == 0.0) {
                eigsrt(d,&v);
                return;
            }
            if (i < 4)
                tresh=0.2*sm/(n*n);

            Initialize to the identity matrix.

            Initialize b and d to the diagonal
            of a.
            This vector will accumulate terms
            of the form  $\tau a_{pq}$  as in equa-
            tion (11.1.14).

            Sum magnitude of off-diagonal
            elements.

            The normal return, which relies
            on quadratic convergence to
            machine underflow.

            On the first three sweeps...
```

```

else
    tresh=0.0;                                     ...thereafter.
for (ip=0;ip<n-1;ip++) {
    for (iq=ip+1;iq<n;iq++) {
        g=100.0*abs(a[ip][iq]);
        After four sweeps, skip the rotation if the off-diagonal element is small.
        if (i > 4 && g <= EPS*abs(d[ip]) && g <= EPS*abs(d[iq]))
            a[ip][iq]=0.0;
        else if (abs(a[ip][iq]) > tresh) {
            h=d[iq]-d[ip];
            if (g <= EPS*abs(h))
                t=(a[ip][iq])/h;                 $t = 1/(2\theta)$ 
            else {
                theta=0.5*h/(a[ip][iq]); Equation (11.1.10).
                t=1.0/(abs(theta)+sqrt(1.0+theta*theta));
                if (theta < 0.0) t = -t;
            }
            c=1.0/sqrt(1+t*t);
            s=t*c;
            tau=s/(1.0+c);
            h=t*a[ip][iq];
            z[ip] -= h;
            z[iq] += h;
            d[ip] -= h;
            d[iq] += h;
            a[ip][iq]=0.0;
            for (j=0;j<ip;j++)                    Case of rotations  $0 \leq j < p$ .
                rot(a,s,tau,j,ip,j,iq);
            for (j=ip+1;j<iq;j++)                  Case of rotations  $p < j < q$ .
                rot(a,s,tau,ip,j,j,iq);
            for (j=iq+1;j<n;j++)                  Case of rotations  $q < j < n$ .
                rot(a,s,tau,ip,j,iq,j);
            for (j=0;j<n;j++)
                rot(v,s,tau,j,ip,j,iq);
            ++nrot;
        }
    }
}
for (ip=0;ip<n;ip++) {
    b[ip] += z[ip];
    d[ip]=b[ip];
    z[ip]=0.0;
    Update d with the sum of  $ta_{pq}$ ,
    and reinitialize z.
}
throw("Too many iterations in routine jacobi");
}
inline void rot(MatDoub_IO &a, const Doub s, const Doub tau, const Int i,
    const Int j, const Int k, const Int l)
{
    Doub g=a[i][j];
    Doub h=a[k][l];
    a[i][j]=g-s*(h+g*tau);
    a[k][l]=h+s*(g-h*tau);
}
};

```

Note that the above routine assumes that underflows are set to zero. On machines where this is not true, the program must be modified. See §1.5.4 and/or find out about the `fesetenv` (Linux) or `__controlfp` (Microsoft) functions.

The Jacobi method does not order the eigenvalues itself. We incorporate the following routine to sort the eigenvalues into descending order. The same routine is used in `Symmeig` in the next section. (The method, straight insertion, is N^2 rather

than $N \log N$; but since you have just done an N^3 procedure to get the eigenvalues, you can afford yourself this little indulgence.)

eigen_sym.h

```
void eigsort(VecDoub_IO &d, MatDoub_IO *v=NULL)
```

Given the eigenvalues `d[0..n-1]` and (optionally) the eigenvectors `v[0..n-1][0..n-1]` as determined by Jacobi (§11.1) or `tqli` (§11.4), this routine sorts the eigenvalues into descending order and rearranges the columns of `v` correspondingly. The method is straight insertion.

```
{
    Int k;
    Int n=d.size();
    for (Int i=0; i<n-1; i++) {
        Doub p=d[k=i];
        for (Int j=i; j<n; j++)
            if (d[j] >= p) p=d[k=j];
        if (k != i) {
            d[k]=d[i];
            d[i]=p;
            if (v != NULL)
                for (Int j=0; j<n; j++) {
                    p=(*v)[j][i];
                    (*v)[j][i]=(*v)[j][k];
                    (*v)[j][k]=p;
                }
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.4.
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[1]
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[2]

11.2 Real Symmetric Matrices

As already mentioned, the optimum strategy in most cases for finding eigenvalues and eigenvectors is, first, to reduce the matrix to a simple form, only then beginning an iterative procedure. For symmetric matrices, the preferred simple form is tridiagonal.

Here is a routine based on this strategy that finds all eigenvalues and eigenvectors of a real symmetric matrix. It is typically a factor of about five faster than the Jacobi routine of the previous section. The implementations of the functions `tred2` and `tqli` that reduce the matrix to tridiagonal form and then find the eigensystem are discussed in the next two sections.

There are two user interfaces, implemented as two constructors. The first constructor is the usual one:

```
Symmeig s(a);
```